

Experiment No. – 1

Aim –

(A.)A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings.

- Every customer must choose **exactly 3 different toppings** to prepare a pizza.
- Write a PostgreSQL query to generate **every possible 3-topping pizza combination** along with its **total ingredient cost**.

Requirements

- Every pizza must contain **exactly 3 different toppings**.
- The same topping **cannot appear more than once** in a pizza.
- Different arrangements of the same toppings should be considered the **same pizza**.
- Display the topping names separated by commas.
- Display the total ingredient cost of each pizza.
- Sort the output:
 - First by **highest total cost**
 - If two pizzas have the same cost, sort them alphabetically.

(B.)Amazon wants to identify **returning customers** who made another purchase shortly after their first purchase.

- Write a PostgreSQL query to find all users who made **another purchase within 1 to 7 days** of an earlier purchase.

Requirements

- Compare purchases **made by the same user only**.
- Ignore comparisons where the same transaction is matched with itself.
- The second purchase must occur **after** the first purchase.
- Same-day purchases must **not** be considered.
- The second purchase must occur within **7 days** of the first purchase.
- Display only the **unique User IDs** that satisfy the above conditions.
- Sort the output in ascending order of user_id.

Tools Required

- Postgres
- PgAdmin4

Queries:

A.

```
CREATE TABLE pizza_toppings (  
    topping_name VARCHAR(50) PRIMARY KEY,  
    ingredient_cost NUMERIC(4,2) NOT NULL  
);  
  
INSERT INTO pizza_toppings (topping_name, ingredient_cost)  
VALUES  
('Pepperoni', 0.50),  
('Sausage', 0.70),  
('Chicken', 0.55),  
('Onions', 0.25),  
('Extra Cheese', 0.40);
```

```
SELECT  
    CONCAT(t1.topping_name, ' ', t2.topping_name, ' ', t3.topping_name) AS pizza,  
    (t1.ingredient_cost + t2.ingredient_cost + t3.ingredient_cost) AS total_cost  
FROM pizza_toppings t1  
JOIN pizza_toppings t2  
    ON t1.topping_name < t2.topping_name  
JOIN pizza_toppings t3  
    ON t2.topping_name < t3.topping_name  
ORDER BY  
    total_cost DESC,  
    pizza ASC;
```

B.

```
CREATE TABLE amazon_transactions (  
    id INT PRIMARY KEY,  
    user_id INT NOT NULL,  
    item VARCHAR(50),  
    created_at TIMESTAMP NOT NULL,  
    revenue INT  
);  
INSERT INTO amazon_transactions (id, user_id, item, created_at, revenue)  
VALUES  
(1, 101, 'biscuit', '2026-03-01 10:00:00', 12),  
(2, 101, 'milk', '2026-03-01 14:00:00', 5),  
(3, 101, 'bread', '2026-03-05 09:00:00', 8),  
(4, 102, 'banana', '2026-03-10 12:00:00', 4),  
(5, 102, 'apple', '2026-03-25 11:00:00', 6),  
(6, 103, 'milk', '2026-03-15 08:00:00', 5),  
(7, 103, 'bread', '2026-03-16 08:00:00', 7);
```

```
SELECT DISTINCT t1.user_id  
FROM amazon_transactions t1  
JOIN amazon_transactions t2  
ON t1.user_id = t2.user_id  
AND t2.created_at > t1.created_at  
AND t2.created_at <= t1.created_at + INTERVAL '7 days'  
ORDER BY t1.user_id;
```

Outputs:

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

```
SELECT
    CONCAT(t1.topping_name, ', ', t2.topping_name, ', ', t3
    (t1.ingredient_cost + t2.ingredient_cost + t3.ingredient
FROM pizza_toppings t1
JOIN pizza_toppings t2
    ON t1.topping_name < t2.topping_name
JOIN pizza_toppings t3
    ON t2.topping_name < t3.topping_name
ORDER BY
    total_cost DESC,
    pizza ASC;
```

Data Output

Messages

Notifications

Showing rows: 1 to 10

Page No: 1

	pizza text	total_cost numeric
1	Chicken, Pepperoni, Sausage	1.75
2	Chicken, Extra Cheese, Sausage	1.65
3	Extra Cheese, Pepperoni, Saus...	1.60
4	Chicken, Onions, Sausage	1.50
5	Chicken, Extra Cheese, Pepper...	1.45
6	Onions, Pepperoni, Sausage	1.45
7	Extra Cheese, Onions, Sausage	1.35
8	Chicken, Onions, Pepperoni	1.30
9	Chicken, Extra Cheese, Onions	1.20
10	Extra Cheese, Onions, Pepperoni	1.15

<pre>SELECT DISTINCT t1.user_id FROM amazon_transactions t1 JOIN amazon_transactions t2 ON t1.user_id = t2.user_id AND t2.created_at > t1.created_at AND t2.created_at <= t1.created_at + INTERVAL '7 days' ORDER BY t1.user_id;</pre>	Showing rows: 1 to 2 <table><thead><tr><th></th><th>user_id integer</th></tr></thead><tbody><tr><td>1</td><td>101</td></tr><tr><td>2</td><td>103</td></tr></tbody></table>		user_id integer	1	101	2	103
	user_id integer						
1	101						
2	103						

Learning Outcome

Learn to generate all unique combinations of records using self-joins.

Understand how to eliminate duplicate permutations by applying ordered join conditions (<).

Calculate aggregate values (total ingredient cost) for each combination.